kjr.com.au

Agentic Engineering:

How AI Agents Are Rewriting the SDLC - and What That Means for Safety

Dr Kelvin Ross

Australian System Safety Conference 2026, Adelaide, 24 Oct 2024





Dr Kelvin Ross is an entrepreneur, technologist and researcher with over 25 years' experience in software engineering, safety-critical systems and enterprise IT. He began his career in safety-critical software engineering in defence, working on FA-18 airborne radar systems, before completing a PhD in safety-critical systems engineering and spending several years consulting in defence and transportation systems. He then moved into the commercial sector, founding KJR, a mid-tier IT consultancy specialising in software testing and assurance across e-health, public administration, finance, insurance, retail and telecommunications. Kelvin has contributed to several boards and national committees, including IntelliHQ, a healthcare AI innovation NFP, Chair of the Qld AI Hub, ACS national technical committee for AI Safety. He is an Associate Adjunct Professor at Griffith University's Institute for Intelligent and Integrated Systems. Over the past two years he has been applying agentic AI tools hands-on across coding, test automation and delivery, and speaks regularly on AI safety, governance and assurance.

Why this talk: one question

Reason 1: the question I keep coming back to, and everything it draws in

The prompt

"Can we use AI where EN 50716 calls for independence between development and verification?"

Tim McComb, 2026

→

The question underneath it

How do humans stay accountable for builds and releases that agents produce?

...which draws in

Independence

Can verification stay independent of development when both are done by agents?

Human approval

What threatens it: volume, complacency, seeing outcomes rather than the work?

Repeatable evidence

Can evidence stay repeatable when the tools update themselves overnight?

We come back to these in the three takeaways.

KJR est. 1997

Why this talk: what it means for KJR

Reason 2: agentic engineering is both our biggest opportunity and an existential question

120 consultants

30 years in quality assurance

15 years in machine learning

Our strategy, set two years ago: Testing AI · AI in Testing

Opportunity

- Assuring AI systems for clients (Testing AI)
- Agents that test faster and deeper (AI in Testing)

Existential

- Agents now write and test code: testing as we know it changes
- So does the day-to-day work of our 120 consultants

Our response

bench-time R&D into future capability

Kushai
our visual-intelligence spin-out, built with agentic engineering

Agent test frameworks
testing agents and agentic pipelines

Agentic test planning
agents that help plan what to test

Agentic test design
agents that help design the tests

Questions for our own team: How will software testing change? Which other pipelines change next? How do our people refine their skills and stay relevant?

KJR est. 1997

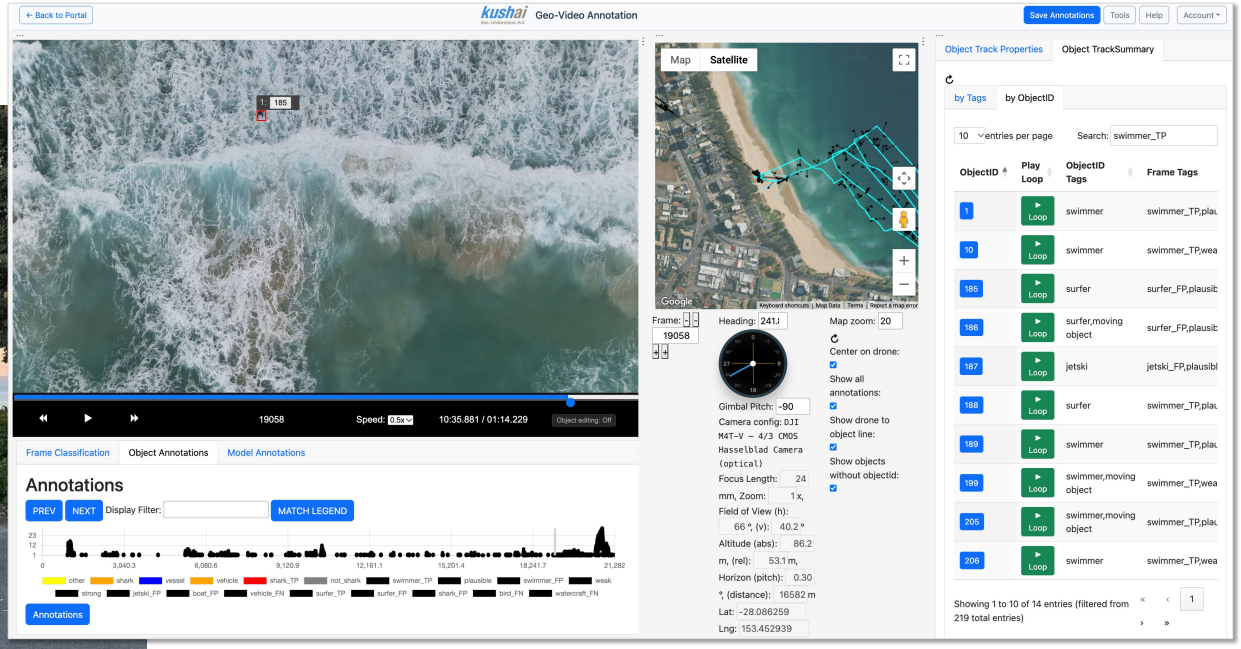
kushai

See. Understand. Act.

Remotely Operated Drone ISR Platforms

www.kushai.ai





est. 1997 KJR

Andrej Karpathy: naming the shifts

The terms we now use for how software is made, and how we build it



Andrej Karpathy

OpenAI founding member, former Director of AI at Tesla, founder of Eureka Labs

What software is

Software 1.0

classic

Rules written by people

Software 2.0

2017

Rules trained by examples

Software 3.0

2025

Rules defined by LLM and context

How we build it

Vibe coding

Feb 2025

Prompt, accept, don't read the code

→

Agentic engineering

2026

Agents build; humans own the architecture, quality and correctness

Photo: Gladwin Analytics, CC BY 3.0, via Wikimedia Commons. Terms: Karpathy, "Software 2.0" (2017); posts on X (Feb 2025, Feb 2026); "Software Is Changing (Again)", YC AI Startup School (Jun 2025).

KJR est. 1997

The biggest shift since waterfall → agile

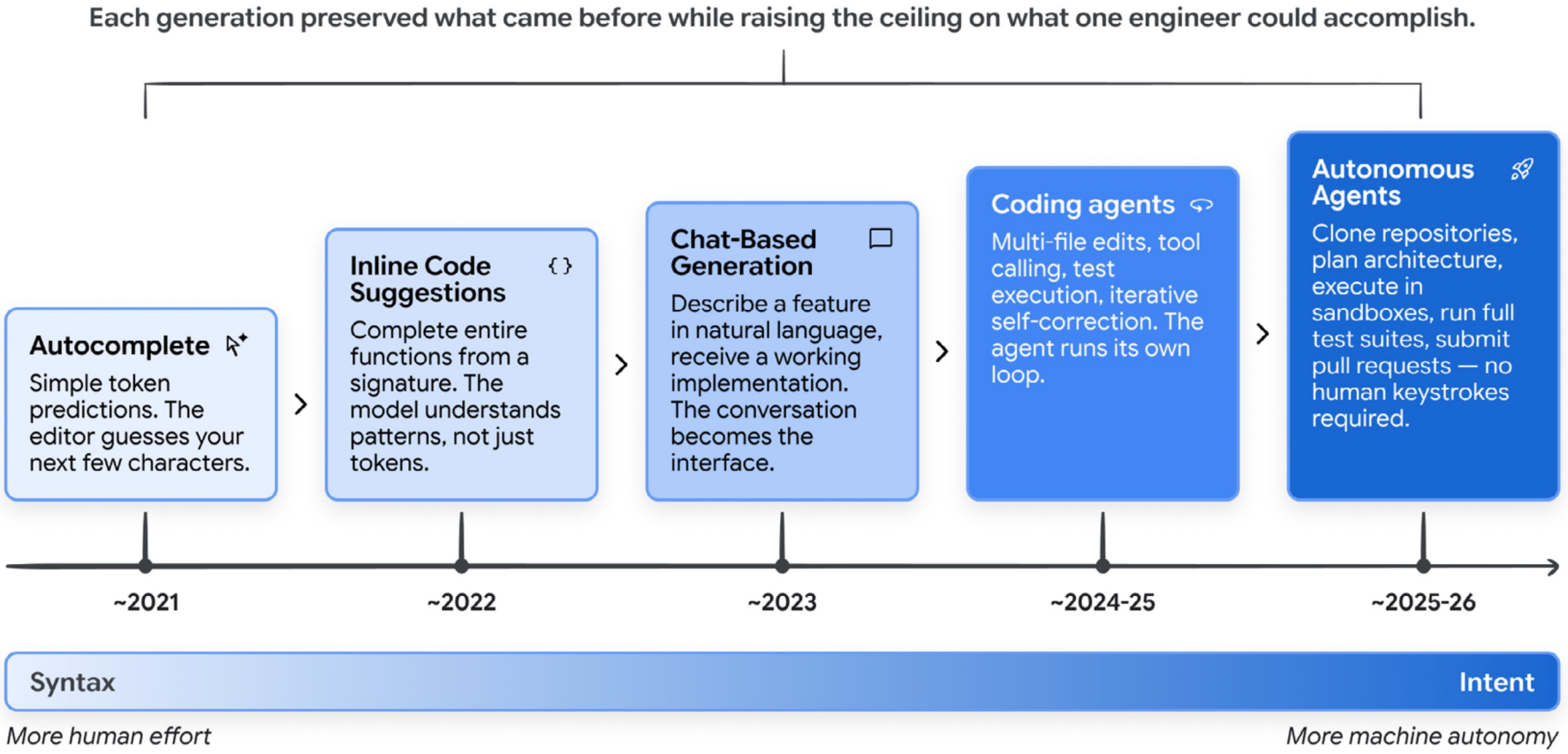
From AI that suggests code to agents that plan, act, test and correct on their own

Autocomplete	Chat assistant	Agent	Multi-agent factory
Suggests the next line **Engineer types**	Writes a function when asked **Engineer asks**	Plans, edits, runs the tests and fixes its own errors **Engineer directs**	Pipelines of agents with skills, reviews and gates **Engineer governs**

	Traditional development	Agentic development
Unit of work	A sprint or phase, measured in weeks	Plan → act → observe → correct loops, in minutes
Who writes the code	Engineers	Agents, directed and reviewed by engineers
Lifecycle	Phases in sequence (V-model, sprints)	Requirements, code and test run together
Where effort goes	Implementation	Requirements, review, verification and audit
Bottleneck	Writing the code	Steering the agents and auditing their output

KJR est. 1997

The biggest shift since waterfall → agile



Source: Osmani, Saboo & Kartakis, "The New SDLC with Vibe Coding", Google/Kaggle, May 2026 (Figure 1)

Vibe coding vs agentic engineering

Not whether you use AI, but how much structure, verification and human judgment surround its output

A spectrum	**Vibe coding**	**Structured AI-assisted**	**Agentic engineering**
How intent is given	Casual prompts	Detailed prompts with examples and constraints	Specs, architecture docs, AGENTS.md
How it's verified	“Does it seem to work?”	Manual testing, spot checks	Automated tests, evals, CI gates
Who reads the code	Often no one	Critical paths reviewed	Humans review the architecture; agent handles detail
When it breaks	Paste the error back in	Human finds the cause, AI fixes it	Agent self-corrects within bounds
Suited to	Prototypes and throwaway code **High risk**	Features in existing codebases **Moderate risk**	Production, team-scale systems **Low risk**

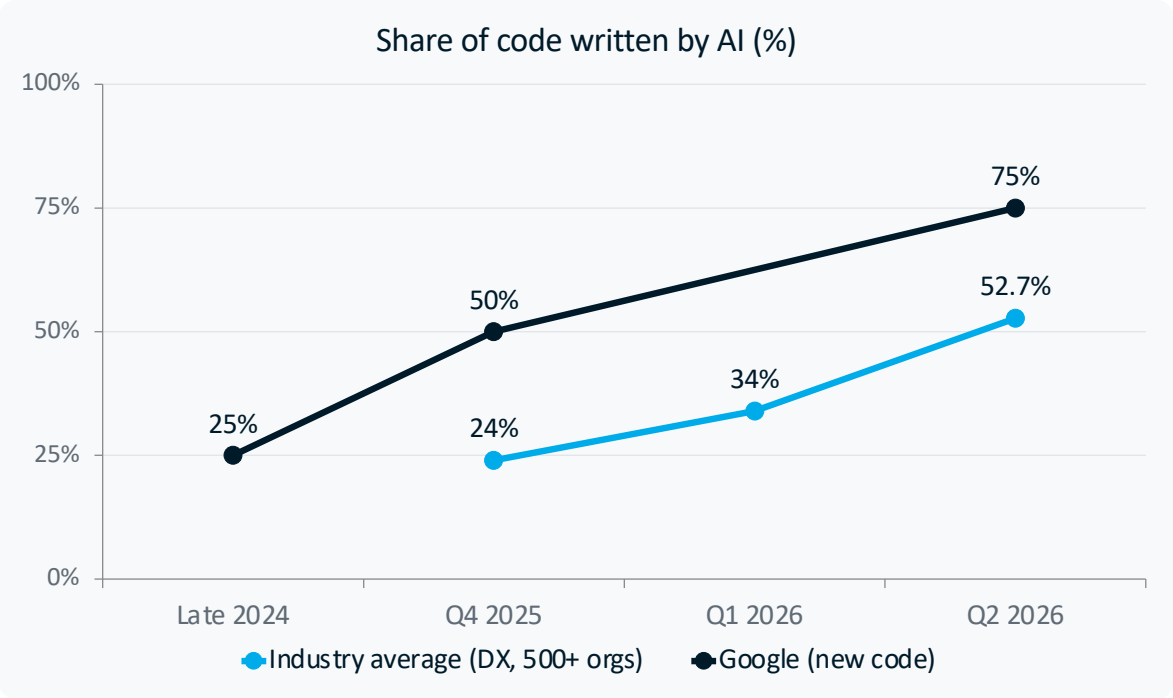
Choose by the stakes. Make the boundary explicit: which projects, branches and environments allow which mode. Safety-critical work sits at the far right, and needs more than even that.

Source: Osmani, Saboo & Kartakis, “The New SDLC with Vibe Coding”, Google/Kaggle, May 2026 (Table 1)

KJR est. 1997

How fast it’s happening

AI-written code passed half of all code in 2026, and agents went mainstream



14% → 37%

of developers use AI agents daily

2025 → Apr 2026 · Stack Overflow

90%

of professional developers use coding agents at work every week

May–Jul 2026 · JetBrains

~28×

growth in median AI spend in one year (tech sector)

Q2 2026 · DX

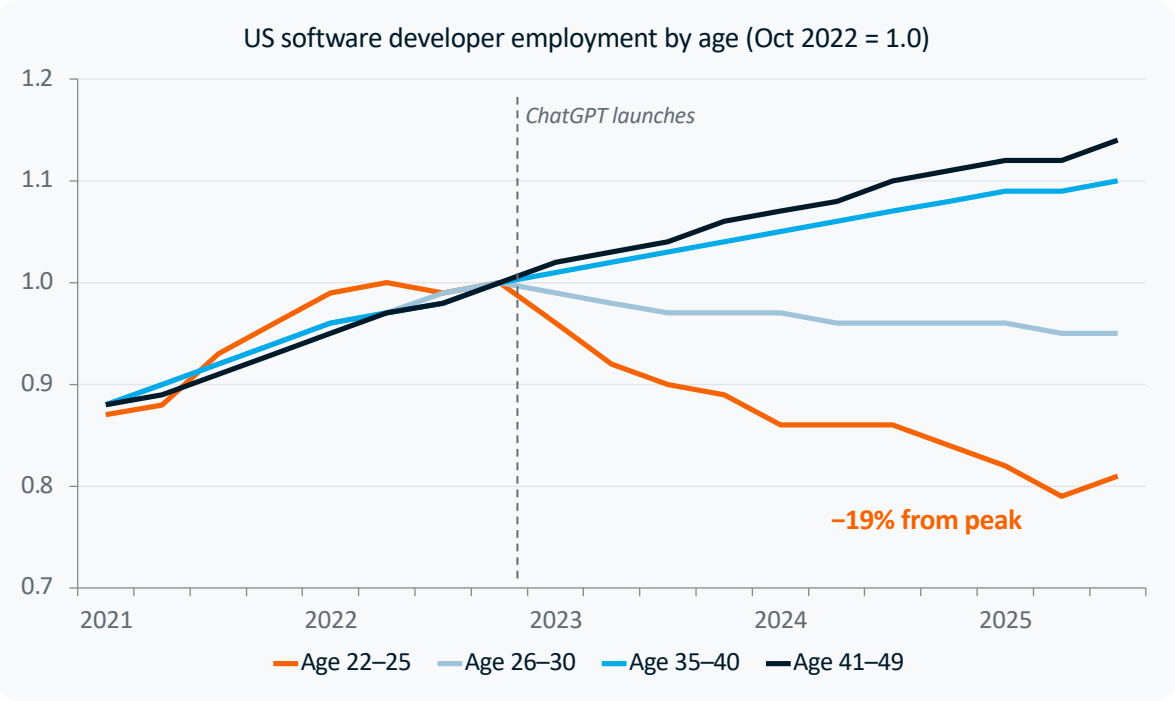
Daily agent use by industry (Apr 2026): fintech 55% · software 44% · **transport 41% · manufacturing 38% · healthcare 36% · government 35% · energy 17%**

Sources: DX Q2 2026 State of AI Impact in Engineering (500+ orgs; telemetry and survey); Google, 22 Apr 2026 (self-reported); Stack Overflow Developer Survey 2025 and Apr 2026 pulse survey; JetBrains Developer Ecosystem Survey 2026. Measures differ between sources.

KJR est. 1997

Losing junior engineers

Agents now do the work juniors learned on, and fewer people are entering the pipeline



−21%

new ICT university students in Australia, 2025 → 2026 (≈9,750 → 7,686)

ACTAC via ACS Information Age, Feb 2026

2.9%

of new Australian university students chose ICT, in a record year for commencements

ACTAC via ACS Information Age, Feb 2026

−15%

Australian graduate job postings in 2025; 35% below the 2023 peak

Indeed Hiring Lab, Apr 2026

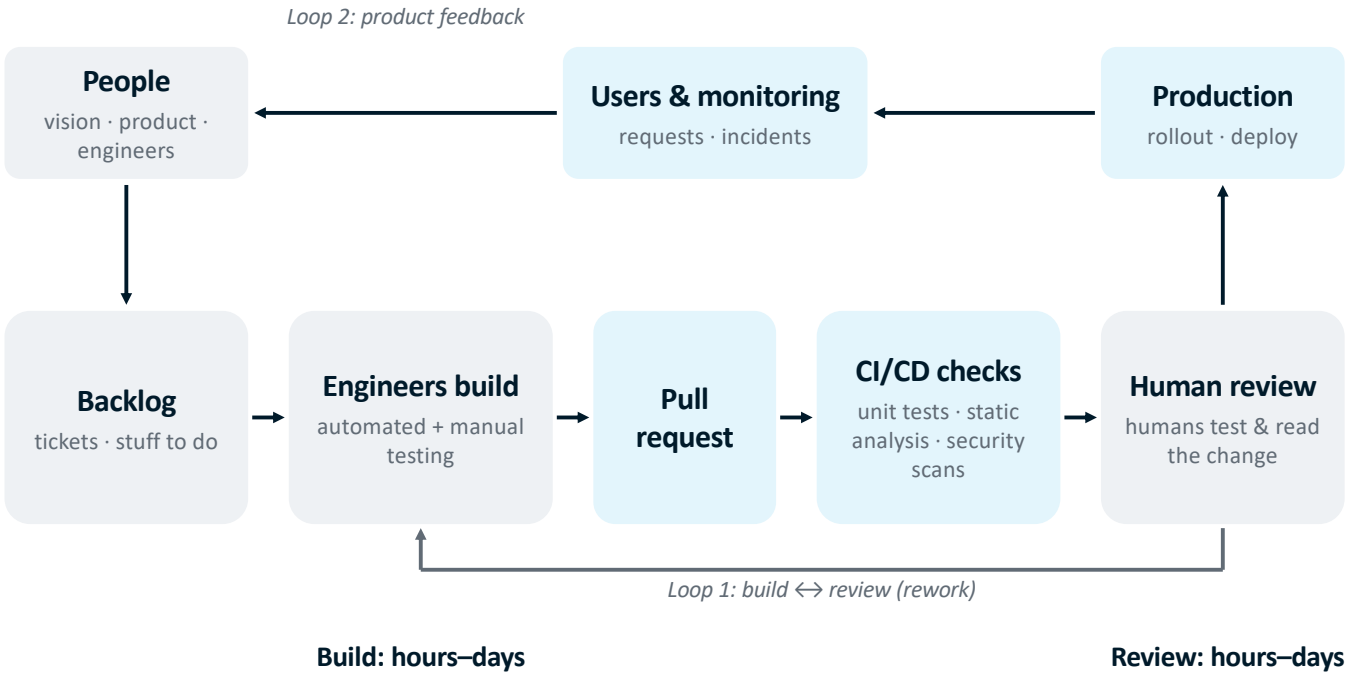
The hollowing out: if agents do the junior work, where do tomorrow's senior engineers, reviewers and safety assessors come from?

Sources: Brynjolfsson, Chandar & Chen (2025), "Canaries in the Coal Mine", Fig. 1, ADP payroll data (approximate, digitised via L. Voss, AI Engineer 2026); ACTAC/ACS Information Age, 3 Feb 2026; Indeed Hiring Lab, Apr 2026.
The Tech Council (2026) finds no disproportionate early-career drop in Australian tech employment yet.

KJR est. 1997

The software factory, circa 2022

How the software factory changed 1/6 · *humans build, humans review*



What changed

- Two loops: product feedback (outer) and build ↔ review (inner)
- Humans write the code and humans review it, at roughly the same pace
- "Software factory" isn't new: the term dates from the 1968 NATO conference

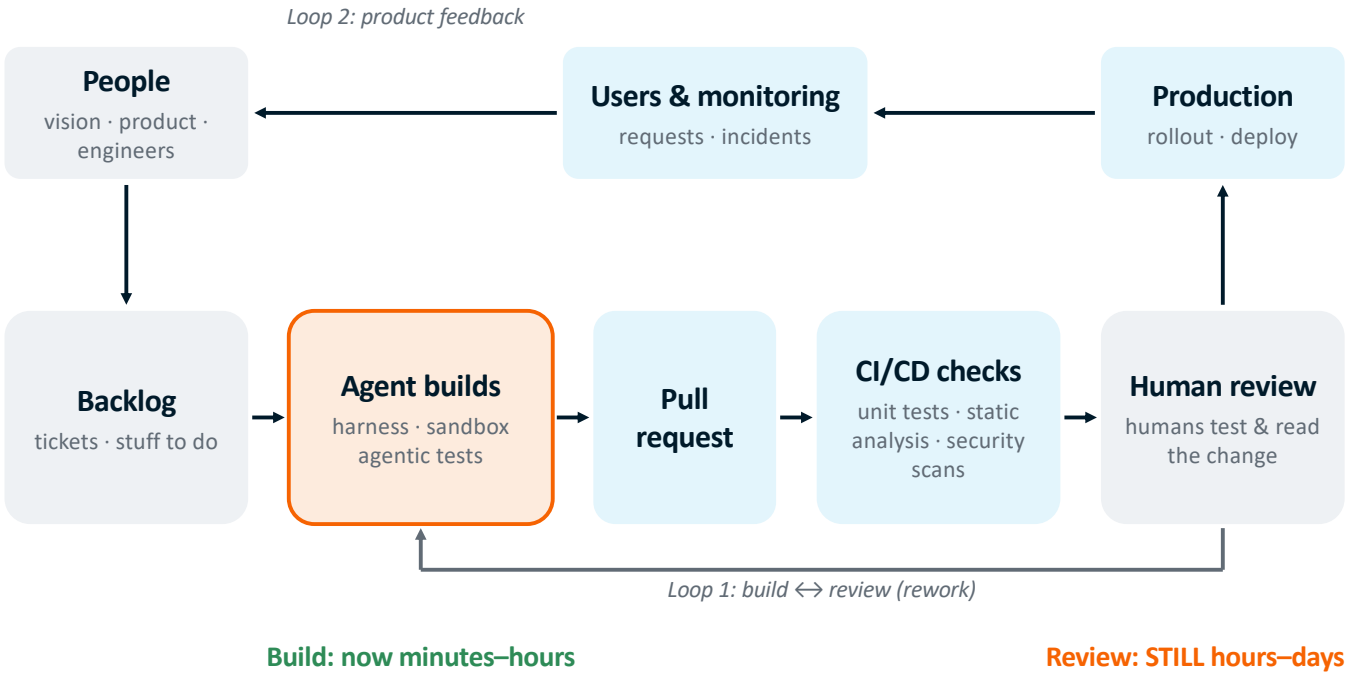
For safety

- Verification independence comes from different people doing the build and the review

Adapted from Dex Horthy (HumanLayer), "Fighting Code Slop: The State of Software Factories", Agentic AI Foundation, 2026

The agentic software factory

How the software factory changed 2/6 · *agents build, humans still review*



What changed

- An agent, inside a harness, replaces the human builder
- Building drops from hours–days to minutes–hours; review doesn't
- Human review becomes the bottleneck

For safety

- Review load scales with agent output, so the pressure to skim grows

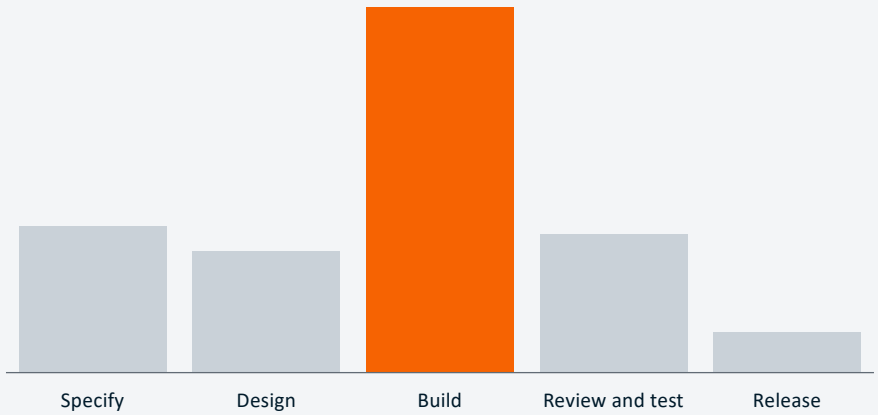
Adapted from Dex Horthy (HumanLayer), "Fighting Code Slop: The State of Software Factories", Agentic AI Foundation, 2026

KJR est. 1997

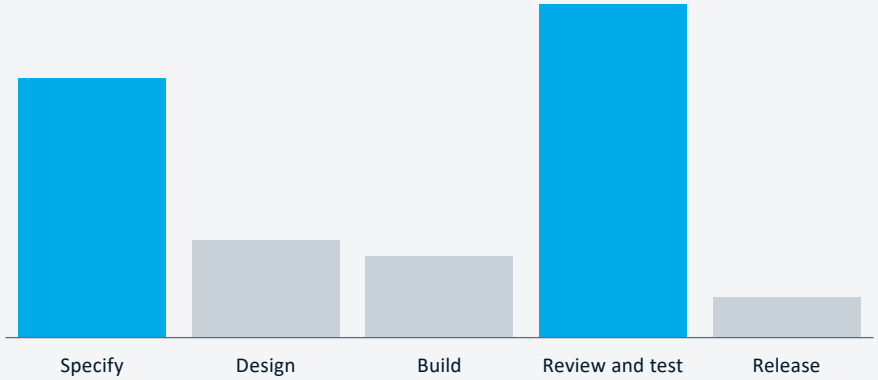
The effort has moved

From writing code to specifying it and reviewing it

Before agents *effort peaks at building*



With agents *effort moves to both ends*



Illustrative

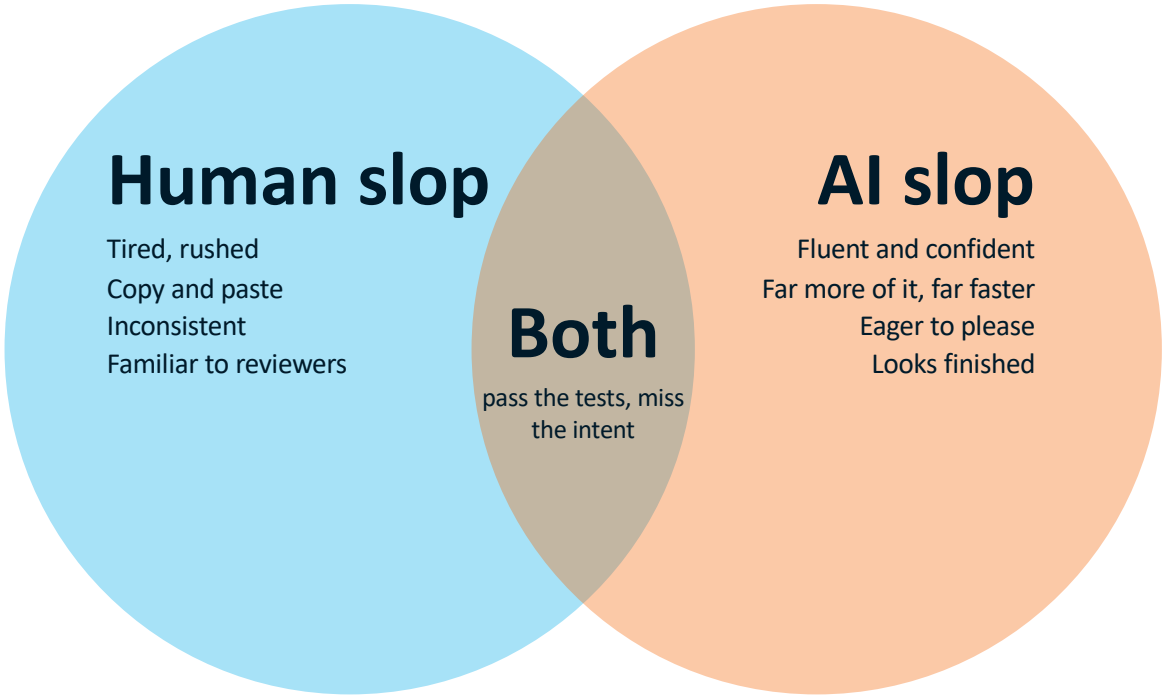
We've made building fast.
The effort moved to specifying and reviewing, and the review load grows with every change an agent makes.

Open question
Have we reduced the burden of overseeing change, or just moved it onto the reviewer?

KJR est. 1997

AI slop vs human slop

Is it the same problem we've always had, just faster?



How do we actively evaluate?

- Review against the spec, not the diff
- Ask "what would make this wrong?"

How do we not become complacent?

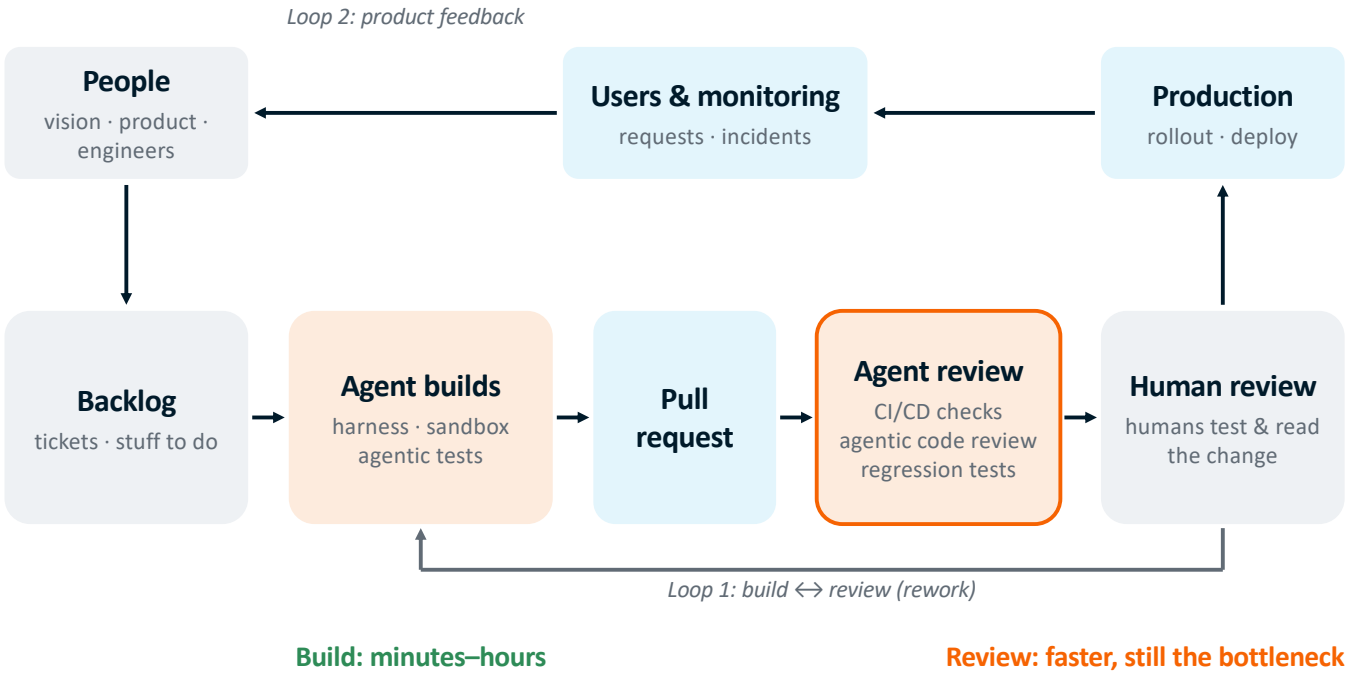
- The better it gets, the less we look
- Seed known defects: do reviewers find them?

Discussion: if we treat agents as teammates, what level of review would we accept from a human doing the same work?

KJR est. 1997

Agents review the agents

How the software factory changed 3/6 · *add agentic code review and regression testing*



Build: minutes–hours

Review: faster, still the bottleneck

What changed

- Agentic code review and regression testing run before a human looks
- Humans review a pre-filtered change, so it's faster
- But humans are still the slowest step

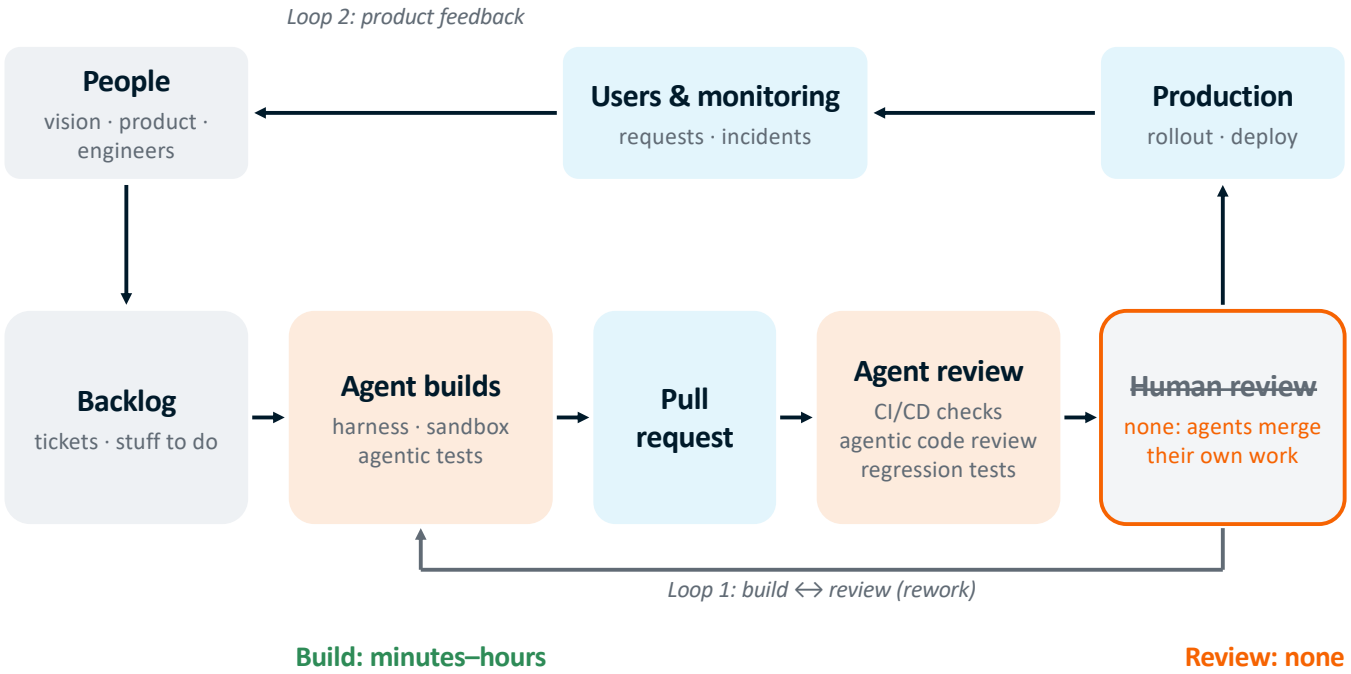
For safety

- A review agent on the same model shares the builder's blind spots

Adapted from Dex Horthy (HumanLayer), "Fighting Code Slop: The State of Software Factories", Agentic AI Foundation, 2026

The lights-off factory

How the software factory changed 4/6 · *remove the human review*



What changed

- "Dark" factories: agents write, review and merge; humans read none of it
- Horthy: slop accumulates, because agents are trained to pass tests, not to keep code maintainable
- Horthy's own team tried it for about six months, then had to rip out and replace large parts of the system

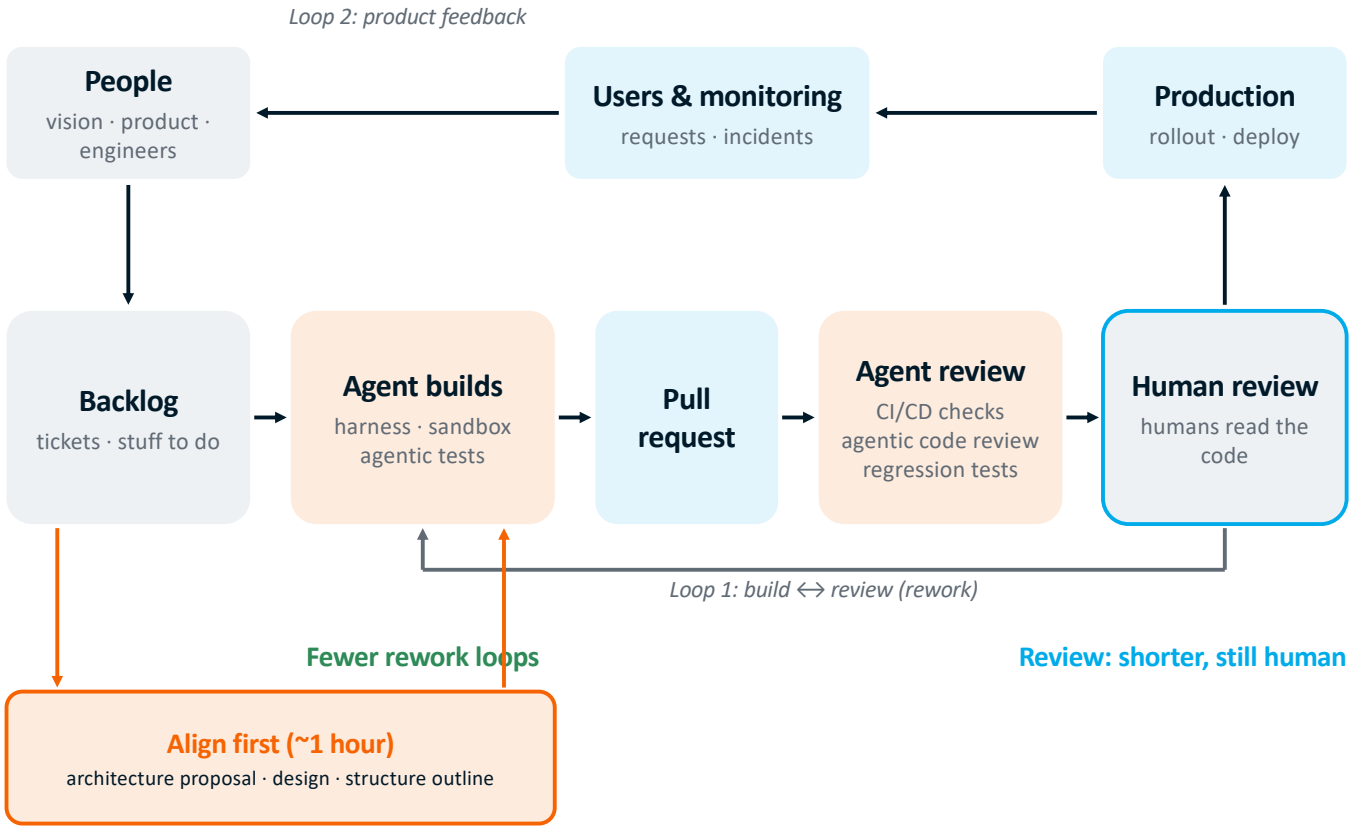
For safety

- No human review, no independent verification, and no one accountable for what ships

Adapted from Dex Horthy (HumanLayer), "Fighting Code Slop: The State of Software Factories", Agentic AI Foundation, 2026

Front-load the humans

How the software factory changed 5/6 · *align before building, keep reading the code*



What changed

- Humans spend about an hour agreeing the design before the agent builds
- Less chance of rework, and a shorter review
- Horthy's advice: keep reading the code, or put the human effort into planning

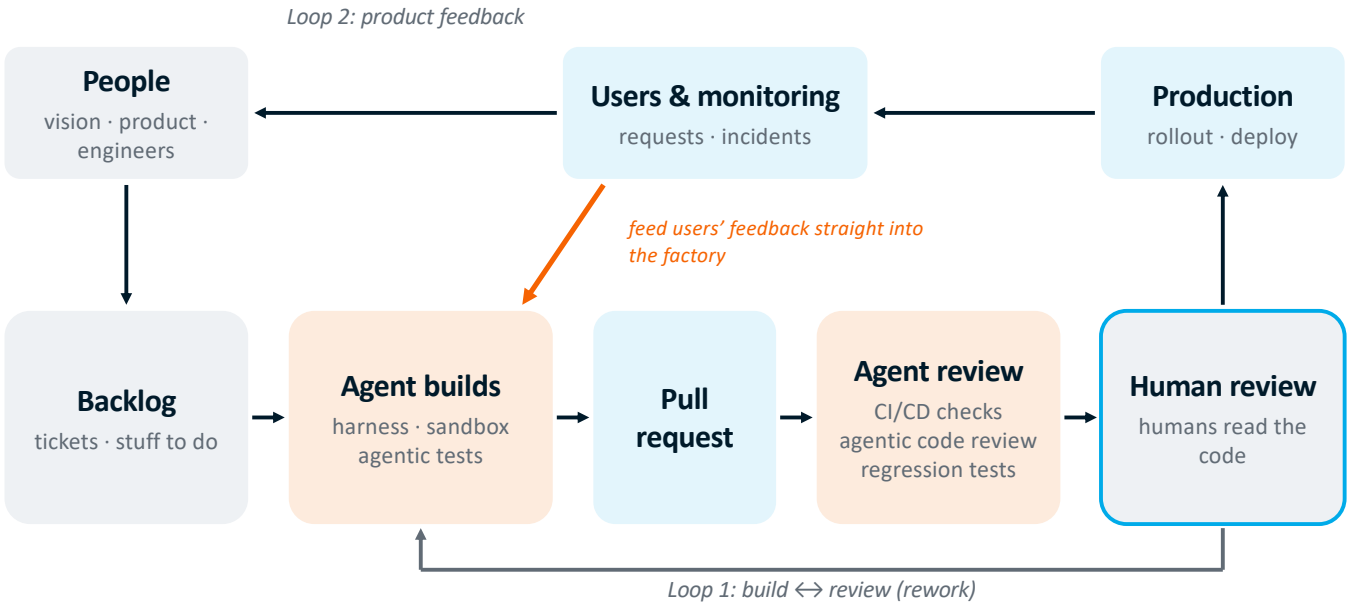
For safety

- Human judgment moves to where it adds most: requirements and design up front, review at the end

Adapted from Dex Horthy (HumanLayer), "Fighting Code Slop: The State of Software Factories", Agentic AI Foundation, 2026

The software forge

How the software factory changed 6/6 · *keep the intent, not just the code*



The forge: plans · specs · design docs · agent sessions · decisions, kept and versioned with the code

What changed

- GitHub centralised the code; an AI-native forge centralises the intent behind it
- Plans, specs and agent sessions stop being scattered across Slack, docs and laptops
- User feedback and incidents flow straight into the factory

For safety

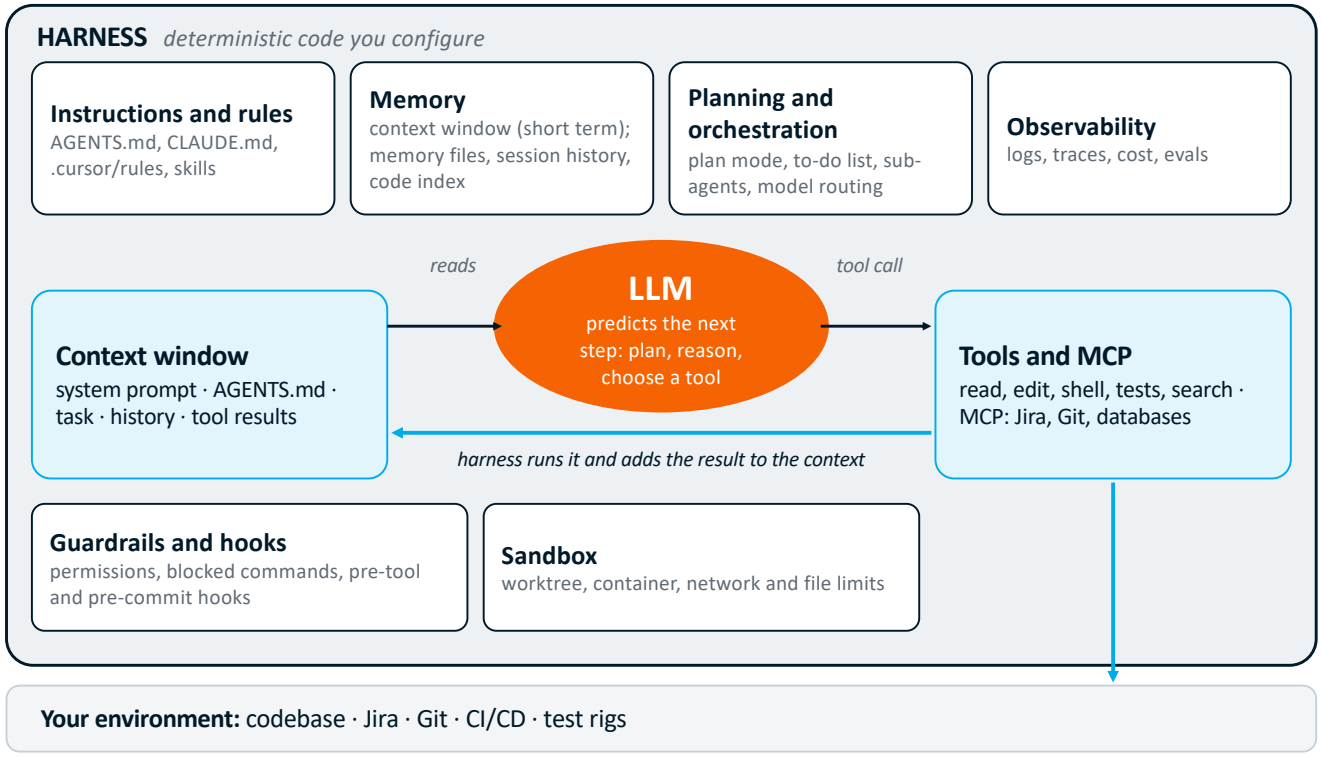
- This is the evidence trail: requirement → plan → session → change → review, kept together

Adapted from Dex Horthy (HumanLayer), "Fighting Code Slop: The State of Software Factories", Agentic AI Foundation, 2026

KJR est. 1997

Agent = model + harness

The LLM only predicts the next step. The harness gives it context, tools, memory and limits, and runs the loop



The model knows the internet, not your system

What the model brings

- A compressed memory of public code and text
- Strong recall: relating and translating patterns
- Fills gaps with the statistically likely answer

What the harness must supply

- Your code, architecture and intent
- Business rules, standards and constraints
- What “done” and “not allowed” mean

Model ~10% · Harness ~90%
of agent behaviour (Day 1 paper)

Probabilistic: the model

Deterministic: harness code you can test and audit

Based on Osmani, Saboo & Kartakis, Google/Kaggle Day 1 paper (May 2026), Fig. 7 “Harness Anatomy”; Weng, “LLM Powered Autonomous Agents” (2023); Anthropic, “Building Effective Agents” (2024)

Context engineering

An agent only knows what's in its context, so the written context is how we steer it

Project *Always loaded*

AGENTS.md, system overview, architecture, coding and safety constraints

Feature *Loaded for the area being changed*

Requirements and design for that area

Task *Loaded for this change*

The ticket, its acceptance criteria and the relevant code

Agent

Starts each session with no memory of the project

↻ **Each change updates the context.** Documents change in the same commit as the code, so the next task starts from an accurate picture.

Steer

Written architecture and constraints keep the agent inside them. Without them, every session makes its own design decisions.

Keep it lean

Long, stale or contradictory context dilutes the agent's attention and costs tokens. Keep the always-loaded layer short.

Control it

Context now shapes what gets built, like a design basis. Review it, version it and control changes to it.

KJR est. 1997

The agentic engineer’s skill set

From writing code to directing, verifying and owning it

Before the agent runs			While it runs	After it runs	
Specify Define intent precisely; resolve ambiguity before the agent fills it	**Decompose and delegate** Right-sized tasks; know what to hand off, and what not to	**Engineer the context** AGENTS.md, skills, tools, guardrails and tests	**Steer and correct** Redirect early; stop and restart when it drifts	**Evaluate and verify** Judge against the spec: evidence, not plausibility	**Integrate** Check changes fit together in intent, not just text

Always

Own the outcome: an accountable human for what ships

Keep the foundations sharp: debugging, design and domain knowledge

Compare: Osmani, Saboo & Kartakis, Google/Kaggle Day 1 paper (May 2026): specification, decomposition, evaluation, system design; Simon Willison, “Vibe engineering” (Oct 2025); Addy Osmani, “Own the Outer Loop” (2026)

KJR est. 1997

Conductor vs orchestrator

Two modes of working with AI agents, and most engineers move between both in the same day

Conductor *Hands-on, real time*

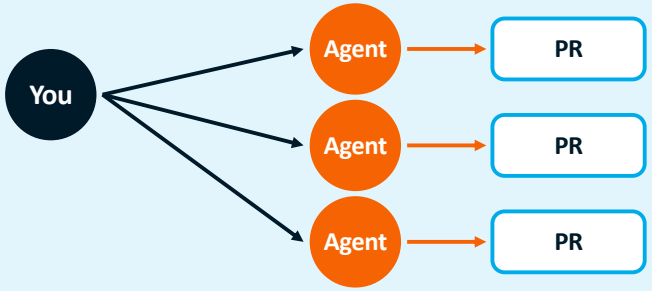


How you work: One AI pair-programmer in the IDE. You watch the code appear and steer it with prompts and corrections.

Best for: Complex logic, tricky debugging, unfamiliar codebases

Strength / risk: Understanding and control, but you become the bottleneck

Orchestrator *Asynchronous, multi-agent*



How you work: Set goals, hand them to agents running in the background (often in parallel), then review the results.

Best for: Well-defined tasks: bug fixes, features on established patterns, migrations, test generation

Strength / risk: Throughput, but you see outcomes, not how the work was done

Orchestration needs different skills: specification, decomposition, evaluation and system design.

Source: Osmani, Saboo & Kartakis, "The New SDLC with Vibe Coding", Google/Kaggle, May 2026 (pp. 31–34)

KJR est. 1997

Who conducts, who orchestrates?

Orchestration lets one person run the whole pipeline, so independence has to be designed back in

	Mainly	Conducts (human judgment)	Orchestrates (delegated)	Independence
Specifier / BA / PO	Conductor	Resolving ambiguity, agreeing acceptance criteria	Drafting stories, edge cases, prototypes	Owns the requirements baseline
Architect	Conductor	Trade-offs and design decisions	Scaffolding, conformance checks	Owns the architecture context
Developer	Orchestrator	Reviewing PRs, design exceptions	Implementation, refactoring, docs	The role being verified
Verifier / QA	Both	Test strategy, oracles, hazard coverage	Test generation, fuzzing, evals	**Independent of development**
Safety assurance	Conductor	Hazard judgments, the safety argument	Traceability, evidence assembly	**Independent; signs off**

Where independence breaks down

- **Role collapse:** one orchestrator's agents write the spec, code and tests (the KJR story)
- **Common mode:** the same model and context write the code and its tests, so a misread requirement passes
- **Rubber stamp:** humans see only outcomes, so approval becomes a formality

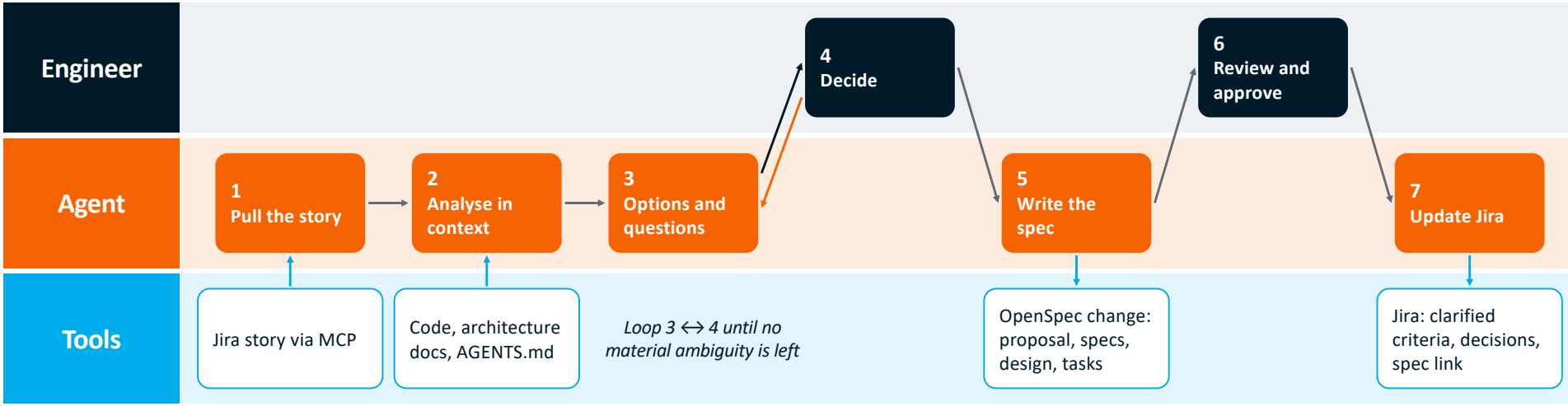
How to keep it

- **Separate human conductors** at each independent decision point
- **Independent context:** verifiers work from requirements and hazards, not the code
- **Diverse models or methods,** enforced by role-scoped permissions in the toolchain

KJR est. 1997

Refine the requirements with the agent

Turn the agent around so it questions you before it builds: the agent analyses and drafts, the human decides



Grill-me in practice *one question at a time, each with a recommended answer*

"Notify the user": email, in-app or both?
Recommended: in-app only, as email isn't configured for this service.

What should happen if the upstream service times out?
Recommended: fail visibly and alert, since the story says no silent failures.

Criterion 2 contradicts the audit rule in the architecture doc. Which wins?
No recommendation: needs the product owner. Logged as an open question.

For safety

- **Ambiguity left in** becomes a silent commission error
- **Decisions and rationale** are recorded: the evidence trail starts here
- **The approved spec** is the oracle for independent verification
- **Watch for anchoring:** recommended answers can make the engineer a rubber stamp

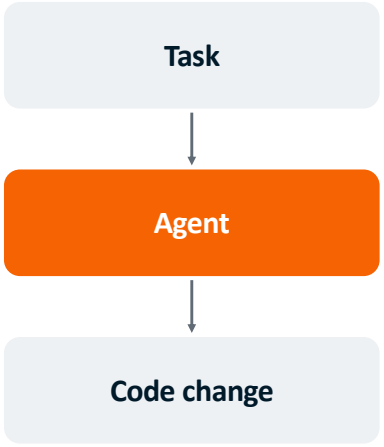
KJR est. 1997

The agent is a coworker, not just a worker

Beyond handing it tasks: conversations that help the human understand the code base

What we assume

the agent as a worker



What it can also be

a coworker you talk to, to understand

Debugging
"Here's the stack trace. Where does it come from?"

Mapping
"Which code handles this feature?"

Onboarding
"Explain this legacy module to a new developer"

Consistency
"Where do the code, design, docs and tests disagree?"

Trade-offs
"What are our options, and what does each cost?"

Simplifying
"How could this be simpler?"

Migration
"What would it take to move this to a new platform?"

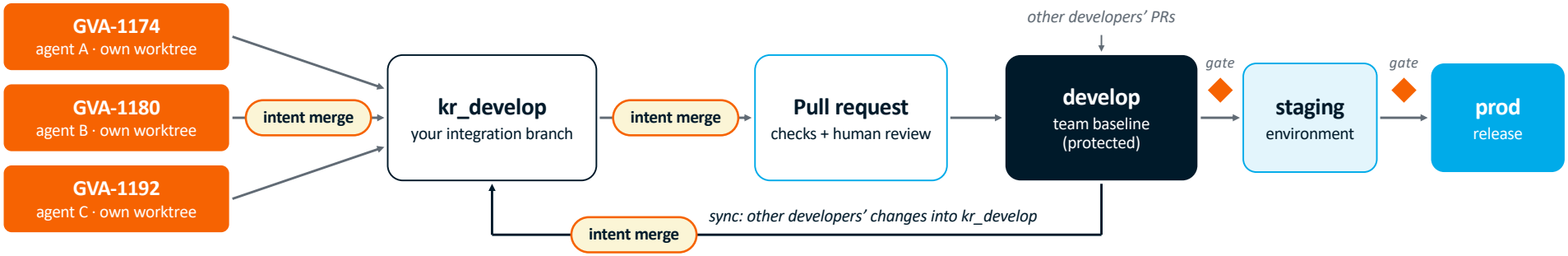
Brainstorming
"What haven't we thought of?"

The human asks the quality questions. Not authoring every line, but understanding every decision.

KJR est. 1997

Fine-grained change control

Agents work in parallel, so every change must be isolated, merged for intent, approved and gated



1 What changed?

- One Jira ticket per branch, small enough to review
- Each agent in its own worktree on that branch
- Commits linked to the story, spec and agent session

2 Does it fit together?

- A clean merge can still conflict in intent
- An agent compares both sides' stories, specs and diffs
- Re-run both tickets' scenarios on the merged result
- Record what interacted and how it was resolved

3 Who approved it?

- Required checks: tests, analysis, security scans
- Code owners route each area to the right reviewer
- Agents assist; an accountable human approves

4 When is it applied?

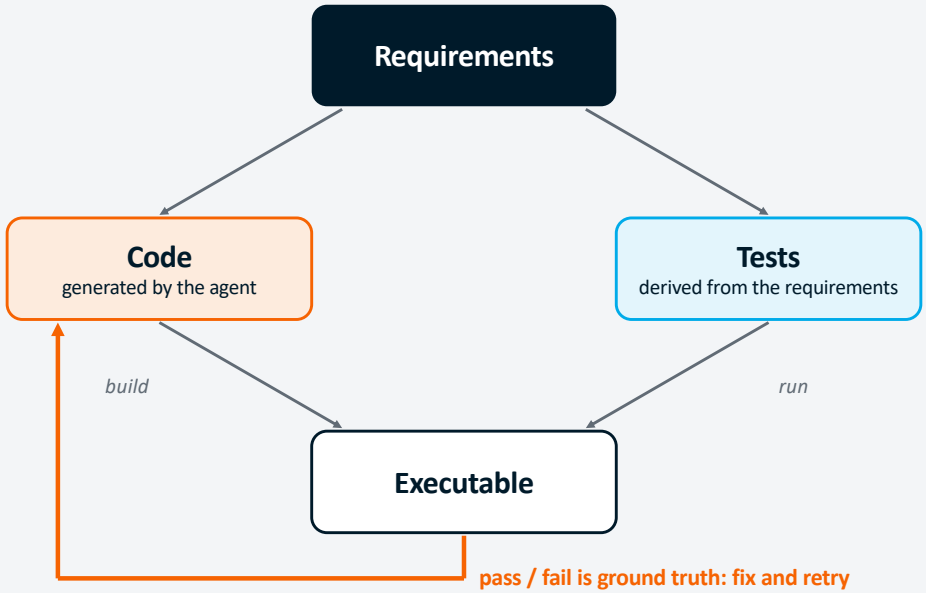
- Nothing goes straight to develop
- Promotion through gated environments
- Every release revertible to a baseline

KJR est. 1997

Build a feedback loop wherever you can

AI improves fastest where its output can be checked. Code can be run; analysis can't

Code: a loop with ground truth



No executable? e.g. hazard analysis

make it checkable, not just reviewable

Consistency

- Every hazard → mitigation → requirement → test
- Flag contradictions across hazard log, FMEA, FTA

Completeness

- Guidewords as a matrix: every cell filled or marked n/a
- Seed known hazards: are they found?

Accuracy

- A second, different model re-derives; compare
- Rule checks, e.g. severity × likelihood → risk

Human review stays for judgment, not for clerical checking that a machine could do.

Jason Wei, "Asymmetry of verification and verifier's law" (2025); "Can LLMs automate the HAZOP process without human intervention?", Safety Science 194 (2026); "Constitutional Meta-STPA", arXiv 2607.08054 (preprint)

KJR est. 1997

Agent Corrects Error

Recent experience in agentic test automation

1. Tester extends the automated tests
2. Tests fail: they find a real defect
3. **Agent "fixes" the product code until the tests pass**
4. Tester checks in the change

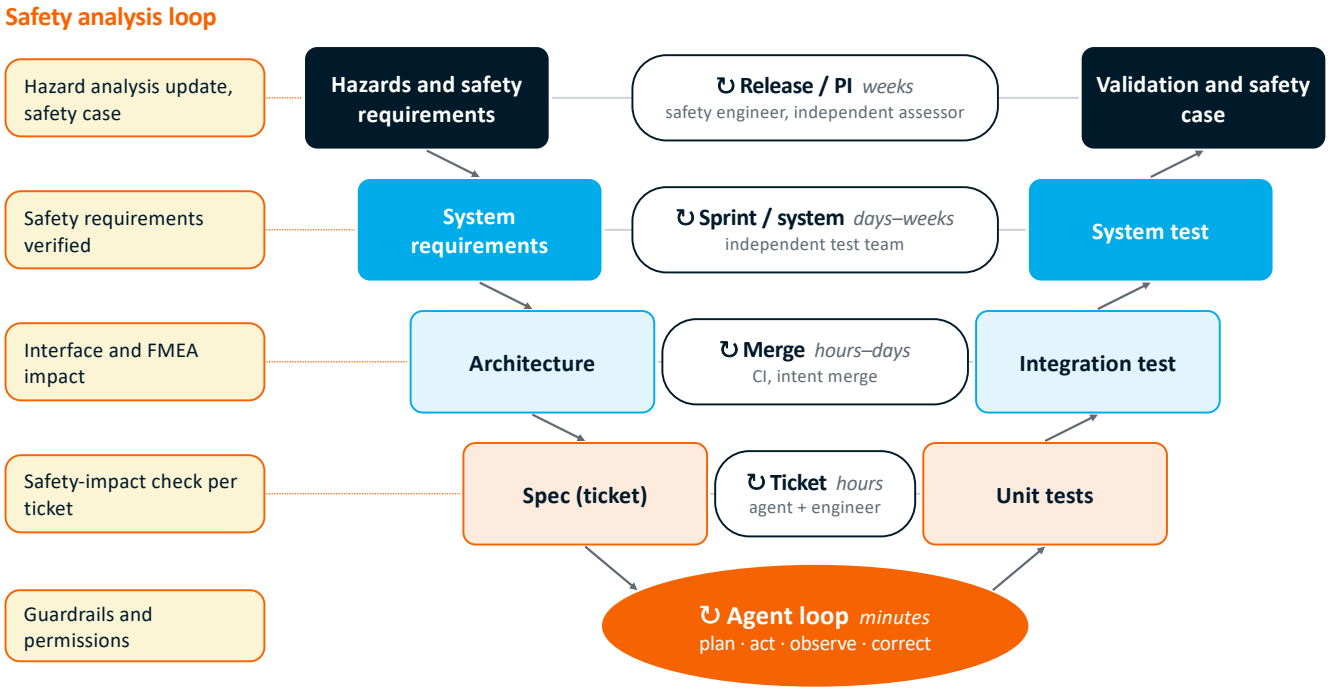
Every step was reasonable.

So where was independence lost, and who should have noticed?

KJR est. 1997

The V runs as nested loops

Agents collapse the bottom of the V into minutes. The upper loops keep their own pace and their independence



How the frameworks map

SAFe (scaled agile)

- Program Increment (8–12 weeks) = release loop; iterations = system loop
- System Team integrates; solution intent holds requirements and traces
- V&V and compliance built into every iteration

SafeScrum (IEC 61508)

- Safety requirements in a separate, linked safety backlog
- Hazard and risk analysis kept outside the sprints
- Evidence assembled in an agile safety case

If one agent runs every loop, the V collapses into one loop with no independence. Keep the outer loops independent while the inner loops speed up.

Sources: SAFe, “Achieving Regulatory and Industry Standards Compliance”; Hanssen, Myklebust & Stålhane, SafeScrum (Springer, 2018), SINTEF papers on SafeScrum and IEC 61508

KJR est. 1997

Who coordinates the loops?

Partly the git pipeline. Three planes keep the loops in step: the work, the artefacts and gates, and the argument

	↻ Agent *minutes*	↻ Ticket *hours*	↻ Merge *hours–days*	↻ Sprint / system *days–weeks*	↻ Release / PI *weeks*
Work *Jira / backlog*	Agent task list	Story, acceptance criteria, safety tag	Linked tickets and dependencies	Sprint goal, system test plan	PI objectives, release scope
backbone **Artefacts and gates** *git pipeline*	**Hooks and local tests in the worktree**	**Branch, PR, required checks, code owners**	**Intent merge, integration tests**	**Deploy to test env, system tests, gate**	**Release tag, baseline, gated promotion**
Argument *safety case*	Guardrail and permission logs	Safety-impact check, trace to hazards	Interface and FMEA impact	Safety requirement verification evidence	Hazard analysis, safety case, independent assessment

Example trigger
coordinated through the pipeline

1. PR touches a safety-tagged requirement or hazard mitigation
2. Pipeline requires a safety-impact assessment and safety-owner review
3. Hazard log updated
4. Evidence linked into the safety case

The pipeline can trigger, gate and record every loop. PI planning, hazard workshops and independent assessment stay human, fed by its evidence.

KJR est. 1997

Four assumptions, revisited

None of them disappears. Each moves somewhere we can design for

	What agents break	Where it lives now
Determinism	The model varies run to run, and changes underneath	**The harness, the tools and deterministic checks**
Repeatable evidence	Re-running the agent won't reproduce the result	**Pinned artefacts: reproduce the check**
Independence	Same model, same spec, same person on both sides	**Scoped context, diverse methods, independent humans**
Accountable human	Volume, speed and complacency erode real review	**Checkpoints with authority, competence, visibility, capacity**

Security cuts across all four: prompt injection or permission misuse can defeat them at once.

KJR est. 1997

Determinism

Assure the product, not the generator: determinism moves to everything around the LLM

Probabilistic

LLM

varies run to run
changes when the vendor updates

Deterministic: where we can rely on it

Harness and tools
deterministic code you configure and test

Deterministic checks
tests on the executable, static analysis, rule validators

Committed artefacts
once built, the executable is deterministic

DO-178C / DO-330 already draws the line

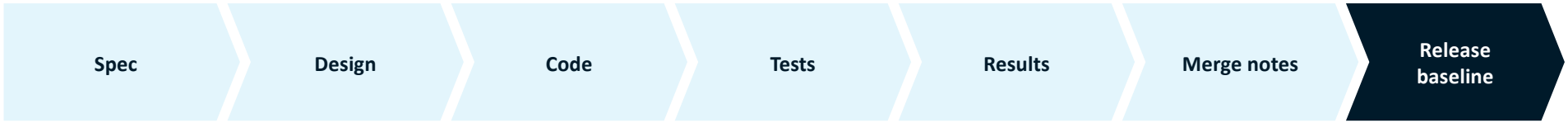
Agent writes the code
Output independently verified → like an unqualified code generator

Agent writes the tests or does the review
Replaces verification → a verification tool: needs qualification-level evidence

Repeatable evidence

Reproduce the check, not the generation: the evidence is the artefacts and the checks we can re-run

Artefacts are the end points of development activities, under configuration management



Pin and record

- Model version and prompts
- AGENTS.md and skills
- Harness configuration
- …as we pin compilers

Records, not procedures

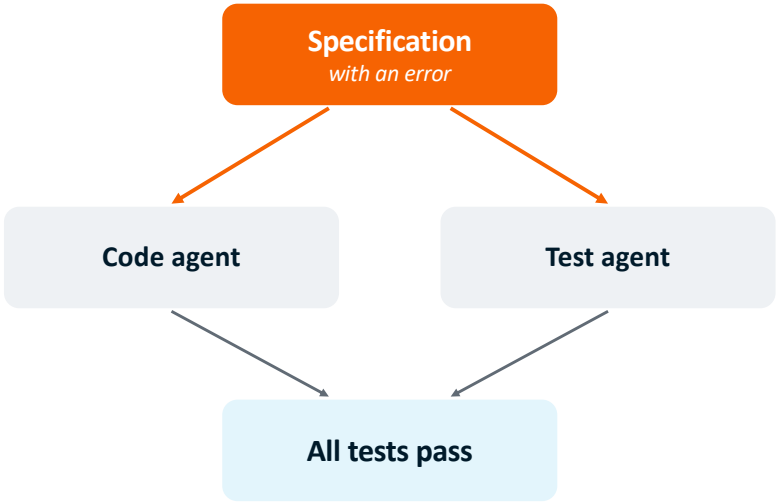
- Session logs explain how it was produced
- They can’t be replayed
- The deterministic checks can

Context is evidence

- The artefacts that steer the agent are the evidence
- Context engineering pays twice

Independence

Removing human coding doesn't remove common mode. It moves it to the specification



Challenge the spec
hazard-driven tests, validation, independent requirements review

Scope the context
the verifier sees requirements, hazards, interfaces, not the code

Diversify
a different model family, and different methods

Independent humans per role
one person orchestrating both sides breaks it (EN 50716)

Honestly: there's no settled answer yet.

An accountable human

Checkpoints aren't enough: the human needs four things to really be accountable

Accountability only works if the human has

Authority
to stop a release

Competence
the skill to judge

Visibility
of what changed, and why

Capacity
the time to really review

Human checkpoints

- Spec approval
- Pull request approval
- Release and safety case sign-off

Human on the loop: watching for gaps

- Escaped defects
- Seeded-defect detection rates
- Review depth and drift

Automation bias: the better it gets, the less we look.

KJR est. 1997

So, is a human accountable?

Back to the opening question

Yes, if we design for it.

No, if we let one agent run the whole loop.

Determinism
in the harness and checks

Repeatable evidence
pinned to artefacts

Independence
scoped context, diverse methods

Accountable human
authority, competence, visibility, capacity

KJR est. 1997

Agentic presentation engineering

This deck was built the way this talk describes, with an AI coding agent

8–10 weeks collecting notes	**3 days** agentic build	**~35** slides drafted	**~30** versioned commits

Collect notes → **Build context** → **Outline and research** → **Steer, review, validate references** → **Build and iterate slides**

What worked

- Technically correct content
- Fast research, with sources to check
- Everything versioned and traceable
- References human validated

What I'd change

- Not my style: padded, too much detail on slides
- Next time: build context and skills for my style
- Refactor to reduce content overload

Under time pressure, I accepted "good enough". That's the capacity and complacency risk, in my own work.

KJR est. 1997

PODCAST SERIES

LISTEN NOW

TRUSTED AI ADOPTION

FROM HYPE TO IMPACT

DRIVING BUSINESS VALUE WITH AI





KJR est. 1997

Key takeaways

1

It’s already in your supply chain

Over half of new code is now AI-written. If you assure software, you’re already assuring AI-written code.

Monday action

Ask suppliers and teams how much code an agent wrote, and how it was checked

2

The value is in the harness, not the model

Build a governed factory of skills, controls and human gates. The effort moves to requirements and verification.

Monday action

Branch protection, PR review and an AGENTS.md wherever agents work

3

Our frameworks assume things agents break

Determinism, independence, repeatable evidence and an accountable human all move. We can design for each, if we start now.

Monday action

Help shape how agentic tools are qualified, before industry practice decides it

KJR est. 1997

Over to you

What would a qualification scheme for agentic development tools look like?

What evidence would you accept?

Do we qualify the harness, not just the model?

How does it keep pace with tools that change monthly?

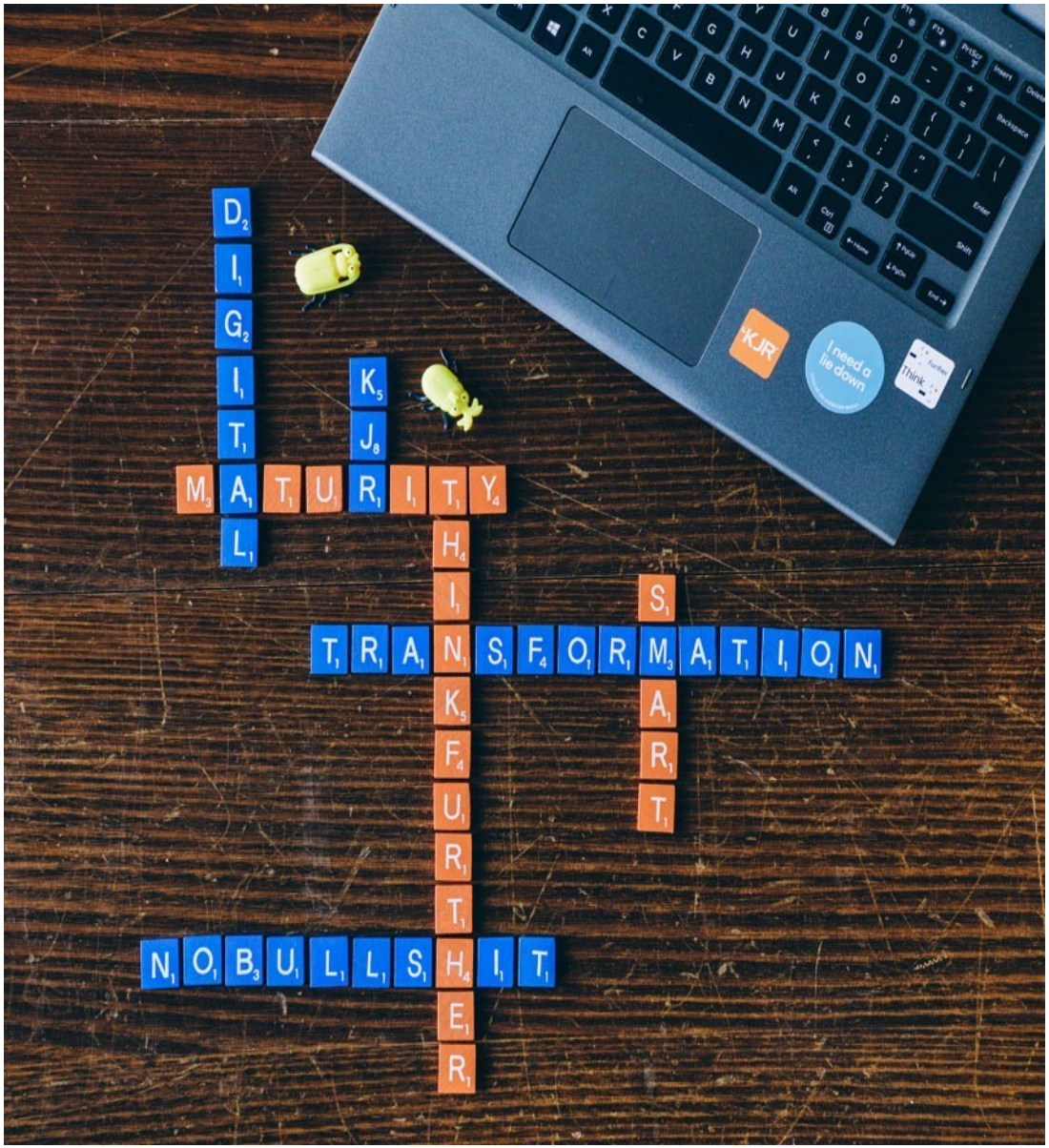
If the safety community doesn't write it, industry practice will.

KJR est. 1997

Trust your AI

Kelvin Ross

kelvin.ross@kjr.com.au



kjr.com.au